

COSC 330 – Final Project Report

Systems Programming in Practice

Name: Nicholas Armenta **Course:** COSC 330 **Due Date:** 11/05/2026

Demo Video

[Insert unlisted YouTube link here]

Part 1 – Interactive Shell with Custom Static & Dynamic Libraries

Summary

Part 1 demonstrates how reusable code is packaged and distributed in C through libraries. The project implements `mylib.c` and `mylib.h`, which expose five operations: addition, subtraction, division, string reversal, and iterative Fibonacci calculation. These are compiled into two forms:

- **Static library (`libmylib.a`)** — created with `ar rcs`, the object code is copied directly into the final executable at link time.
- **Shared (dynamic) library (`libmylib.so`)** — compiled with `-fPIC` and `-shared`, loaded by the OS at runtime via the dynamic linker.

The menu-driven shell (`shell.c`) is linked against the shared library using `-lmylib -Wl,-rpath,.`, so the runtime linker knows where to find `libmylib.so` without requiring `LD_LIBRARY_PATH` to be set manually.

What is a Static Library?

A **static library** (`.a` on Linux, `.lib` on Windows) is an archive of compiled object files bundled together. At link time, the linker copies only the object code your program actually uses into the final executable.

Advantages:

- Self-contained binary — no external dependencies at runtime.
- Slightly faster function calls (no indirection through the PLT/GOT).
- Easier deployment on systems where the library may not be installed.

Disadvantages:

- Larger executables — every program that uses the library carries its own copy.
- Bug fixes or updates require recompiling and relinking every dependent binary.
- Cannot be swapped at runtime without recompiling.

Real-world example: `libc.a` — the C standard library in static form. Used when building statically linked binaries (e.g., `gcc -static`), common in embedded systems or minimal Docker base images.

What is a Dynamic (Shared) Library?

A **shared library** (`.so` — Shared Object — on Linux, `.dll` on Windows) is loaded into a process's address space at runtime by the dynamic linker (`ld.so`). Multiple running processes can share a single copy of the library in memory.

Advantages:

- Smaller executables.
- Library can be updated (patched for security bugs) without recompiling applications.
- Memory-efficient: one copy of the `.so` is mapped into all processes that use it.

Disadvantages:

- "Dependency hell" — the correct version must be present at runtime.
- Slightly more overhead per call (PLT indirection on first call).

.so vs .dll: Both serve the same purpose — runtime-loaded shared libraries — but use different binary formats. `.so` uses the ELF format (Linux/Unix) while `.dll` uses the PE/COFF format (Windows). The linking and loading mechanisms differ (e.g., `dlopen` vs `LoadLibrary`), but the conceptual model is identical.

Real-world examples:

- `libc.so.6` — the GNU C Library, loaded by virtually every Linux process.
- `libssl.so` — OpenSSL's TLS library, shared across browsers, servers, and system tools.
- `libpthread.so` — POSIX threads library, used by any multithreaded application.

Relevance to Systems Programming

Libraries are one of the most fundamental code-reuse mechanisms in systems software. The Linux kernel exposes its API through `glibc`, which ships as a shared library. Understanding the difference between static and dynamic linking is essential for writing portable, efficient, and maintainable C programs.

Part 2 – Understanding systemd

Summary

Part 2 demonstrates how Linux manages long-running processes through `systemd`. A custom unit file (`myshell.service`) was written and installed to run the Part 1 shell program (or a simplified daemon version) as a managed background service.

What is systemd?

systemd is the init system and service manager used by the majority of modern Linux distributions (Ubuntu, Debian, Arch, Fedora, Amazon Linux 2, etc.). It is the first process started by the kernel at boot — it runs as **PID 1** — and is responsible for:

- Bringing up all other system services in the correct order (using dependency declarations).
- Managing the lifecycle of daemons (start, stop, restart, reload).
- Collecting service logs via the **journal** (`journald`), queryable with `journalctl`.
- Handling socket activation, timers (replacing cron for many use cases), and mount points.

Before systemd, Linux used SysV init scripts — plain shell scripts in `/etc/init.d/` that were hard to parallelize and had no uniform logging. systemd replaced this with a declarative, parallel, dependency-aware model.

Structure of a `.service` Unit File

A unit file has three sections:

```
[Unit]
Description=Human-readable name
After=network.target      # ordering: start after networking is up

[Service]
Type=simple              # process type (simple, forking, oneshot,
                           notify)
ExecStart=/path/to/binary # command to run
Restart=on-failure        # restart policy
StandardOutput=journal    # redirect stdout to journald

[Install]
WantedBy=multi-user.target # which boot target enables this service
```

Commands Used

```
# Install and manage the service
sudo cp part2/myshell.service /etc/systemd/system/
sudo systemctl daemon-reload      # reload unit file changes
sudo systemctl start myshell      # start now
sudo systemctl enable myshell     # start on every boot
sudo systemctl status myshell     # show PID, state, recent logs
sudo systemctl stop myshell       # stop now
sudo systemctl disable myshell    # remove from boot

# View logs
sudo journalctl -u myshell        # all logs for this service
sudo journalctl -u myshell -f     # live tail
sudo journalctl -u myshell --since "10 min ago"
```

systemd's Role in Modern Linux

systemd is the backbone of process management in modern Linux. It replaces the fragile, sequential SysV boot process with a parallel, dependency-driven system that can reduce boot times dramatically. On cloud instances like EC2, virtually every background service — SSH, the AWS agent, cron, Docker — is a systemd unit. Understanding systemd is therefore not optional for systems programmers working in cloud environments.

Beyond service management, systemd includes timers (cron replacements), socket activation (start a service only when a connection arrives), and `systemd-networkd/systemd-resolved` for network configuration.

Its centralized journal makes debugging significantly easier compared to scattered log files across `/var/log/`.

Part 3 – EC2-to-EC2 Communication (File Transfer)

Summary

Part 3 implements a TCP client-server file transfer in C. The server listens on port 9000, accepts a single connection, reads an 8-byte big-endian length header, then streams the file body into `received_file1.txt`. The client opens the target file, sends the length header, then streams the file in 64 KB chunks.

Design Decisions

Port choice: Port 9000 is in the unprivileged ephemeral range (above 1024), so it does not require root to bind. It is also unlikely to conflict with common system services.

Addressing: On two EC2 instances in the same VPC, communication uses **private IP addresses** (e.g., `10.0.x.x`). Public IPs would work but add unnecessary latency and may incur data transfer costs. On localhost (for local testing), `127.0.0.1` is used instead.

Data framing: Raw TCP is a byte stream with no built-in message boundaries. Without framing, the receiver cannot know when the file ends. The protocol prefixes every transfer with an 8-byte `uint64_t` (big-endian, network byte order via `htobe64/be64toh`) containing the exact file size. The receiver reads exactly that many bytes and stops.

Error handling: The code handles `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `recv()`, and `fopen()` failures, printing descriptive errors via `perror()` and exiting cleanly.

Checksums: Not implemented in this version. For production use, a SHA-256 of the file could be sent after the data and verified by the receiver to detect corruption in transit.

Security group rules (EC2): Both instances must be in the same security group with an inbound rule allowing TCP on port 9000 from the security group's own ID (source = `sg-xxxxxxx`). This restricts access to instances within the group only.

Relevance to Systems Programming

Socket programming is fundamental to distributed systems. Every network protocol — HTTP, SSH, DNS, gRPC — is built on top of the same `socket/bind/listen/accept/connect/send/recv` primitives used here. Understanding how TCP framing, addressing, and error handling work at the system call level is essential before using higher-level networking abstractions.

Part 4 – EC2-to-S3 Communication

Summary

Part 4 uses Python's `boto3` library to perform four operations against an S3-compatible object store:

- 1. Generate `output.txt` containing the current UTC timestamp, hostname, and OS info.
- 2. Upload `output.txt` to a bucket as the key `output.txt`.
- 3. List the bucket contents, printing each key with its size and last-modified timestamp.
- 4. Download `output.txt` back as `downloaded_output.txt`.

For local development, **MinIO** (a self-hosted S3-compatible server) is used in place of AWS S3. The `boto3` client is pointed at `http://localhost:9000` via `endpoint_url`, and credentials are read from the environment variables `MINIO_ACCESS_KEY` and `MINIO_SECRET_KEY` — never hardcoded in source.

How EC2 Obtains Temporary Credentials (IAM Role)

On a real EC2 instance, no credentials are hardcoded. Instead:

- 1. An **IAM role** with an appropriate policy (e.g., `s3:PutObject`, `s3:GetObject`, `s3:ListBucket`) is created and attached to the EC2 instance.
- 2. The EC2 **Instance Metadata Service (IMDS)** at `http://169.254.169.254/latest/meta-data/iam/security-credentials/<role-name>` exposes temporary `AccessKeyId`, `SecretAccessKey`, and `SessionToken` credentials that rotate automatically every few hours.
- 3. `boto3` calls IMDS automatically when no explicit credentials are provided. The developer writes no credential-handling code at all.

This is fundamentally more secure than hardcoded keys: credentials are short-lived, rotated automatically, scoped to a specific role, and never stored in source code or config files.

IAM Policy Permissions Required

Permission	Purpose
<code>s3:PutObject</code>	Upload a file to the bucket
<code>s3:GetObject</code>	Download a file from the bucket
<code>s3:ListBucket</code>	List objects in the bucket

The principle of least privilege applies: the role is granted only these three actions on the specific bucket ARN, not `s3:*` on all resources.

Relevance to Systems Programming

Cloud storage is now a standard component of production systems. Understanding how IAM, IMDS, and the S3 API interact is as important as understanding the POSIX file API. The credential-rotation model enforced by IAM roles is a direct application of systems security principles: minimize trust surface, use time-limited secrets, and never store credentials in plaintext.

Part 5 – Integration: Libraries, systemd, Sockets, and S3 in Cloud-Native Systems

Modern cloud-native applications combine all four concepts demonstrated in this project:

- **Libraries** package and version business logic (.so files are deployed as layers in container images, updated independently of the application binary).
- **systemd** manages the lifecycle of every service on a Linux host — from the Docker daemon to Nginx to custom microservices. Knowing how to write unit files, manage dependencies between services, and query the journal is a daily operational skill.
- **Sockets** are the foundation of all inter-service communication. HTTP, gRPC, message queues, and database connections are all TCP streams at the kernel level. The framing and error-handling patterns from Part 3 appear in every network protocol implementation.
- **S3 / object storage** has replaced local filesystems for persistent data in cloud architectures. Logs, ML model artifacts, media files, and backups all flow through S3-compatible APIs. The IAM credential model from Part 4 is the standard pattern for zero-credential cloud access.

Together, these four building blocks — libraries, process management, networking, and cloud storage — form the practical foundation of systems programming in a Linux cloud environment.

Report prepared for COSC 330 – Systems Programming in Practice